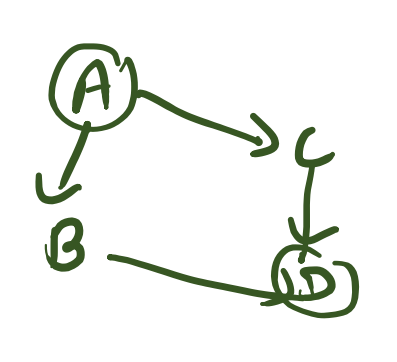
Lec-44

26/08/26 7:06 PM

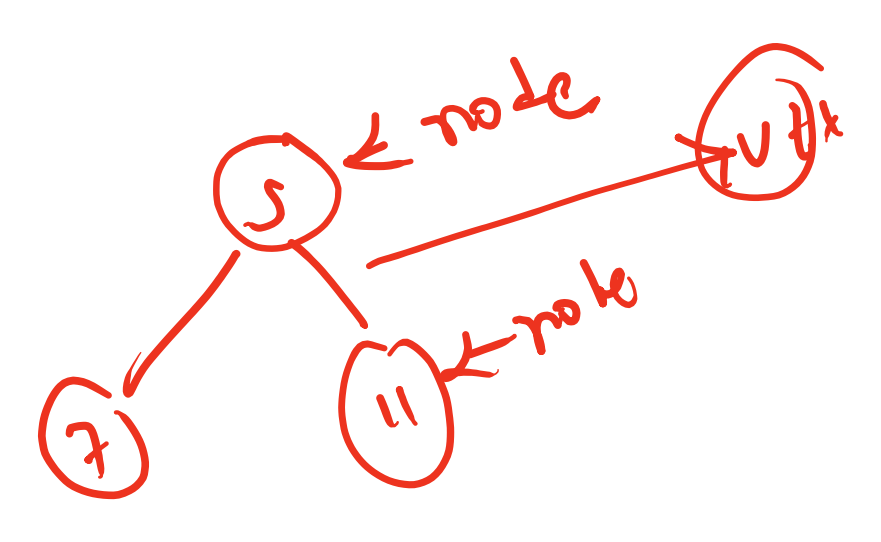
(Graph is finite set of Edge and vtx)



Graph
Issue → map
Router ✓

① Tree → Graph ✓
Graph → Tree ✗

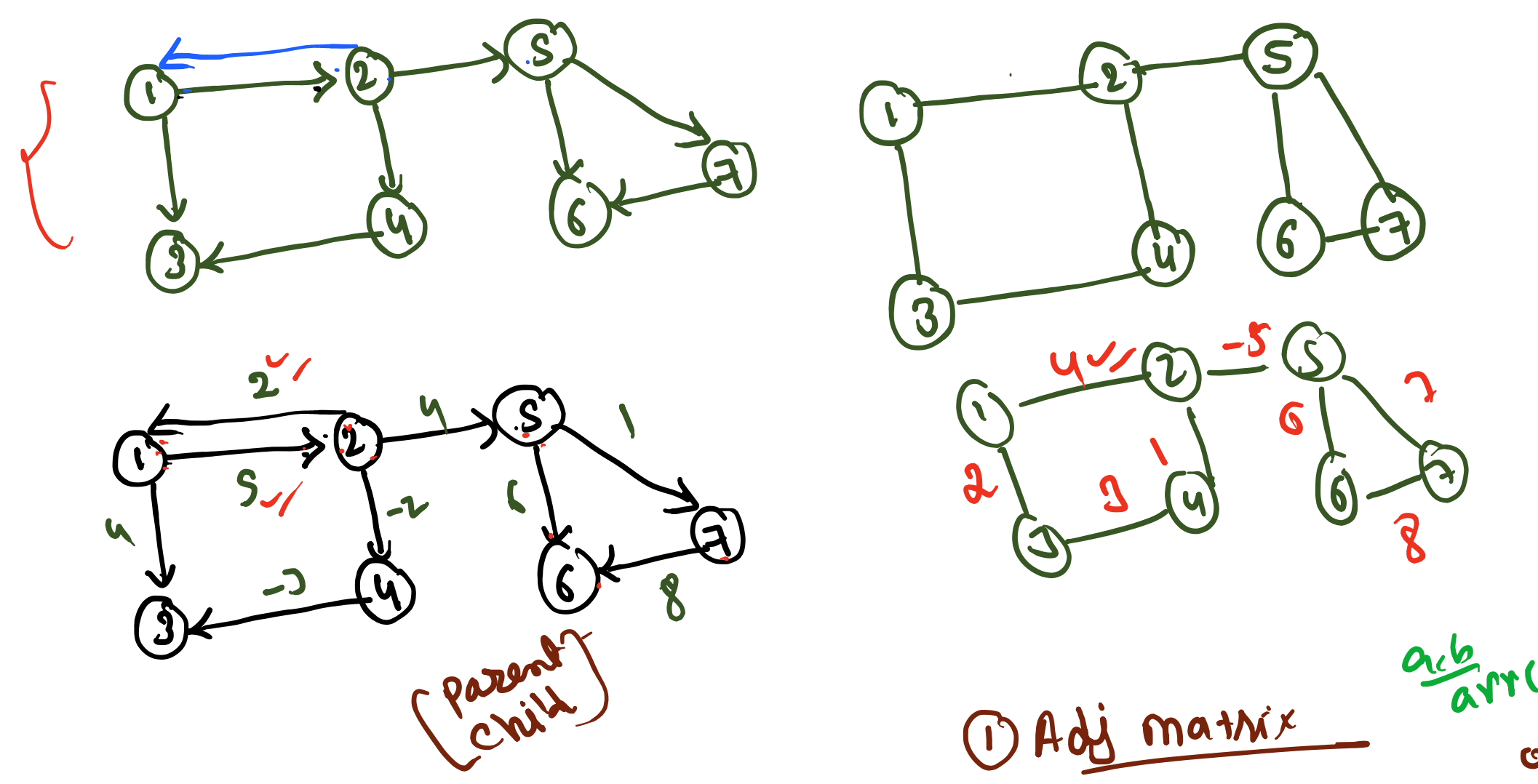
①


node
vtx
node
5
7
11

Type

directed graph — unweight / weight

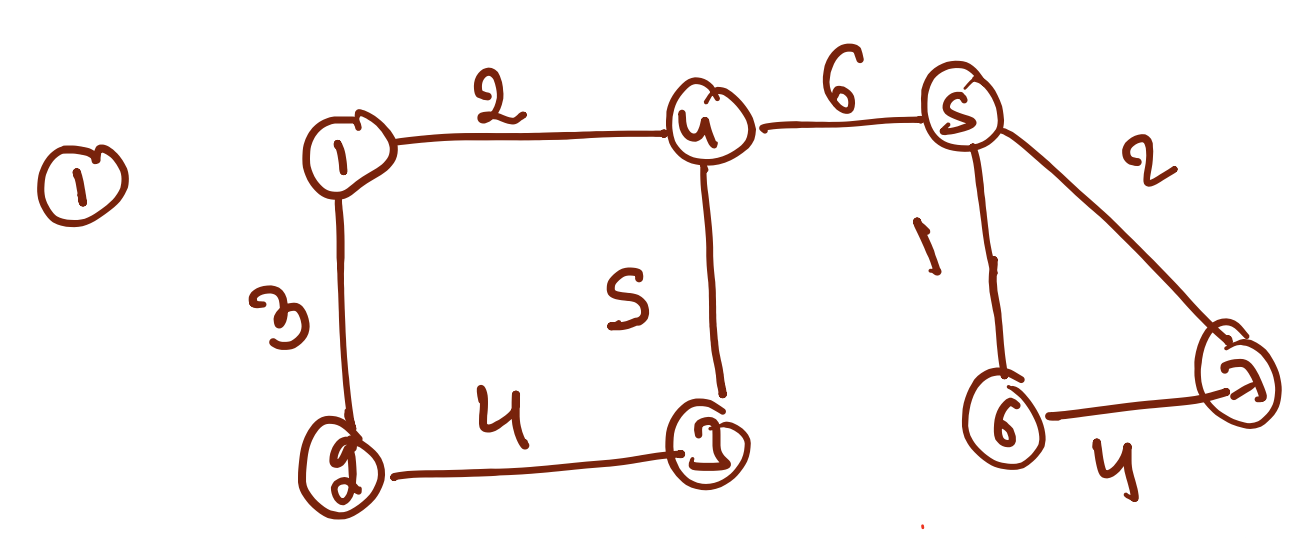
Undirected graph → unweighted → weight


1
2
3
4
5
6
7
2
5
4
-2
-3
1
6
8
parent child
4
2
3
1
-5
6
7
8

① Adj matrix

a(c,b) arr(a)(b) = ✓

O(1)


①
1
2
4
6
5
2
3
5
1
2
3
4
7
6
4

	1	2	3	4	5	6	7
1	0	3	0	2	0	0	0
2	3	0	4	0	0	0	0
3	0	4	0	5	0	0	0
4	2	0	5	0	6	0	0
5	0	0	0	6	0	1	2
6	0	0	0	0	1	0	4
7	0	0	0	0	2	4	0

① Adj List — O(n)

1-7

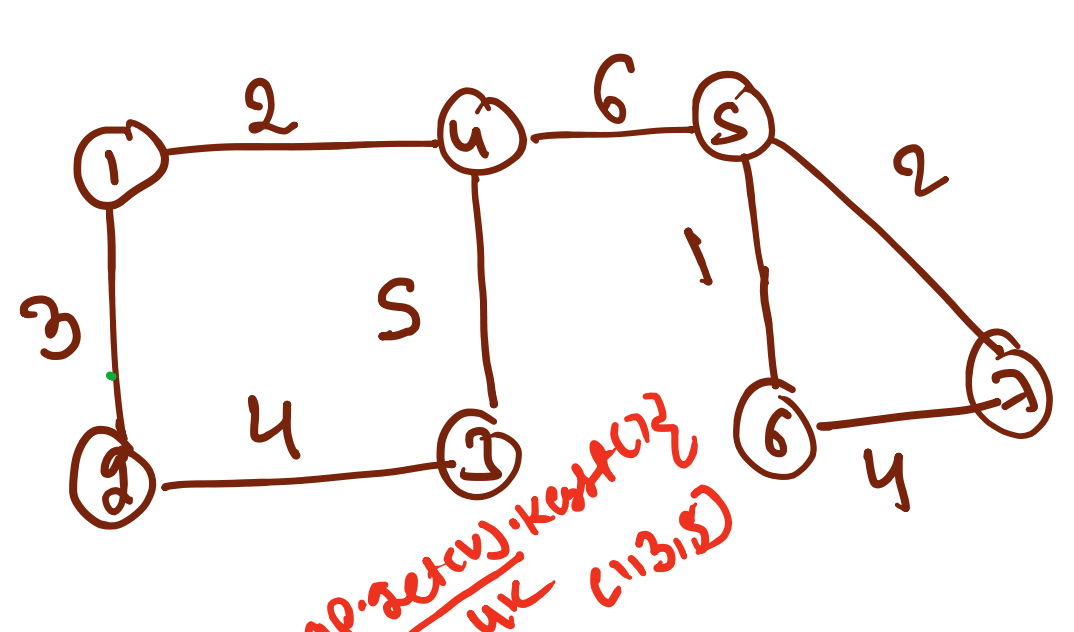
1 → 4|2 → 2|3
2 → 1|3 → 3|4
3 → 2|4 → 4|5
4 → 1|2 → 5|6 → 3|5
5 → 4|6 → 6|1 → 7|2
6 → 5|1 → 7|4
7

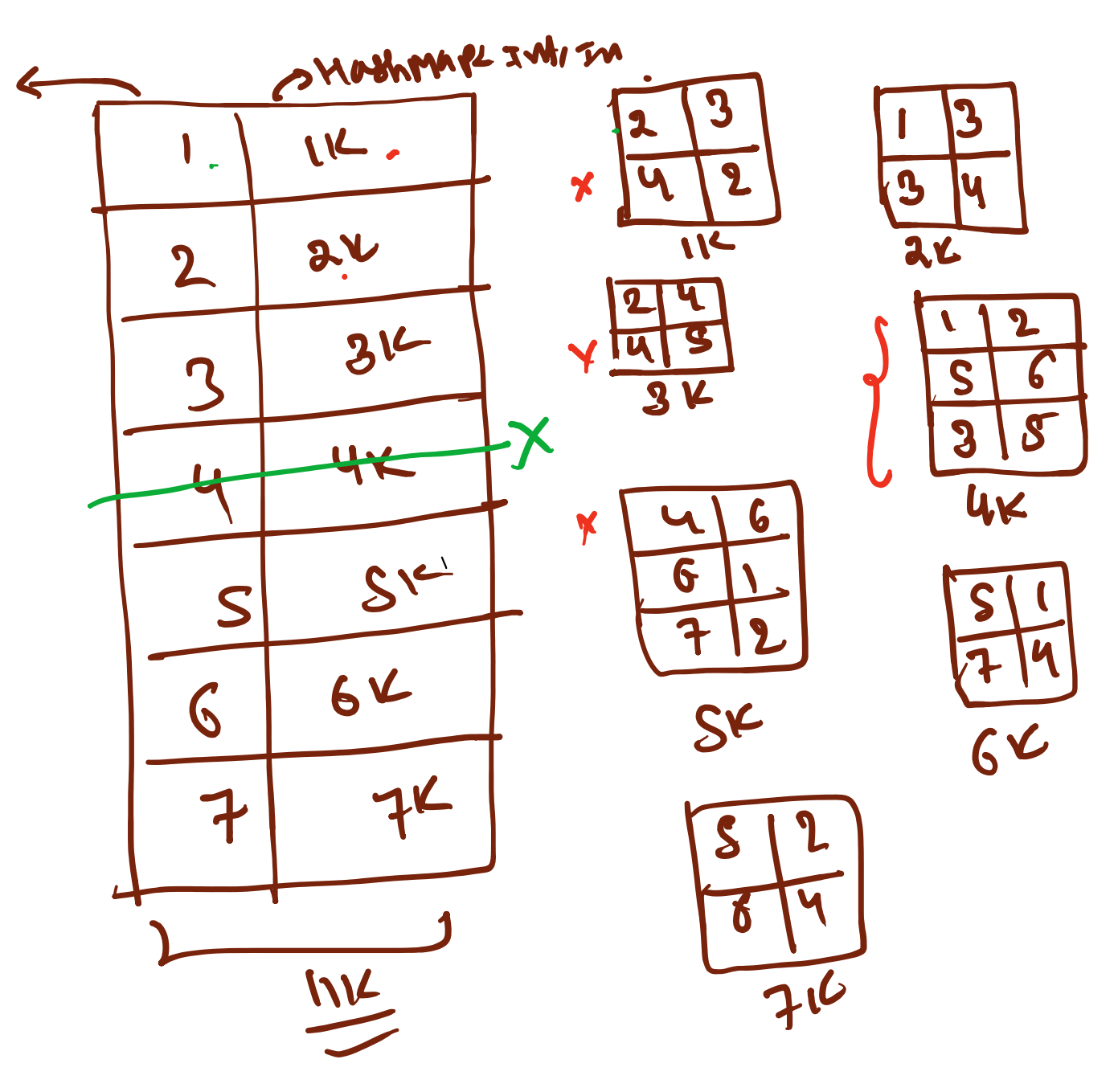
map.get(v1).remove(v2)

map.get(v1).containsKey(v2)

map.get(v1).put(v2, cost)

Int ←

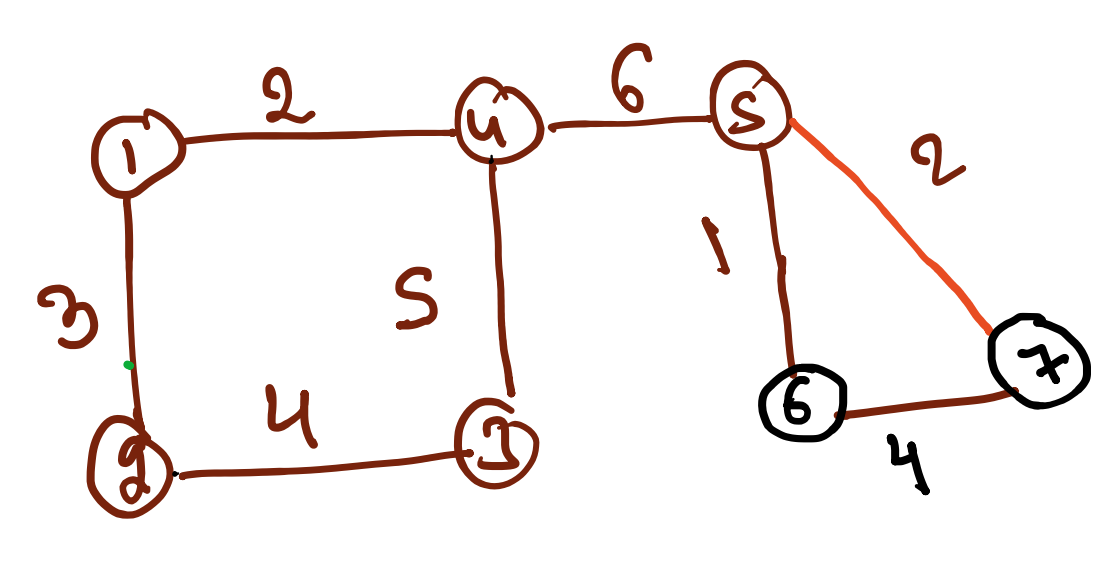

1
2
4
6
5
2
3
5
1
2
3
4
7
6
4


Hashmap Int, Int
1 1K
2 2K
3 3K
4 4K
5 5K
6 6K
7 7K
11K
2 3 4 2
1K
1 3 3 4
2K
2 4 4 3
3K
1 2 5 6 3 8
4K
4 6 6 1 7 2
5K
5 1 7 4
6K
5 2 8 4
7K

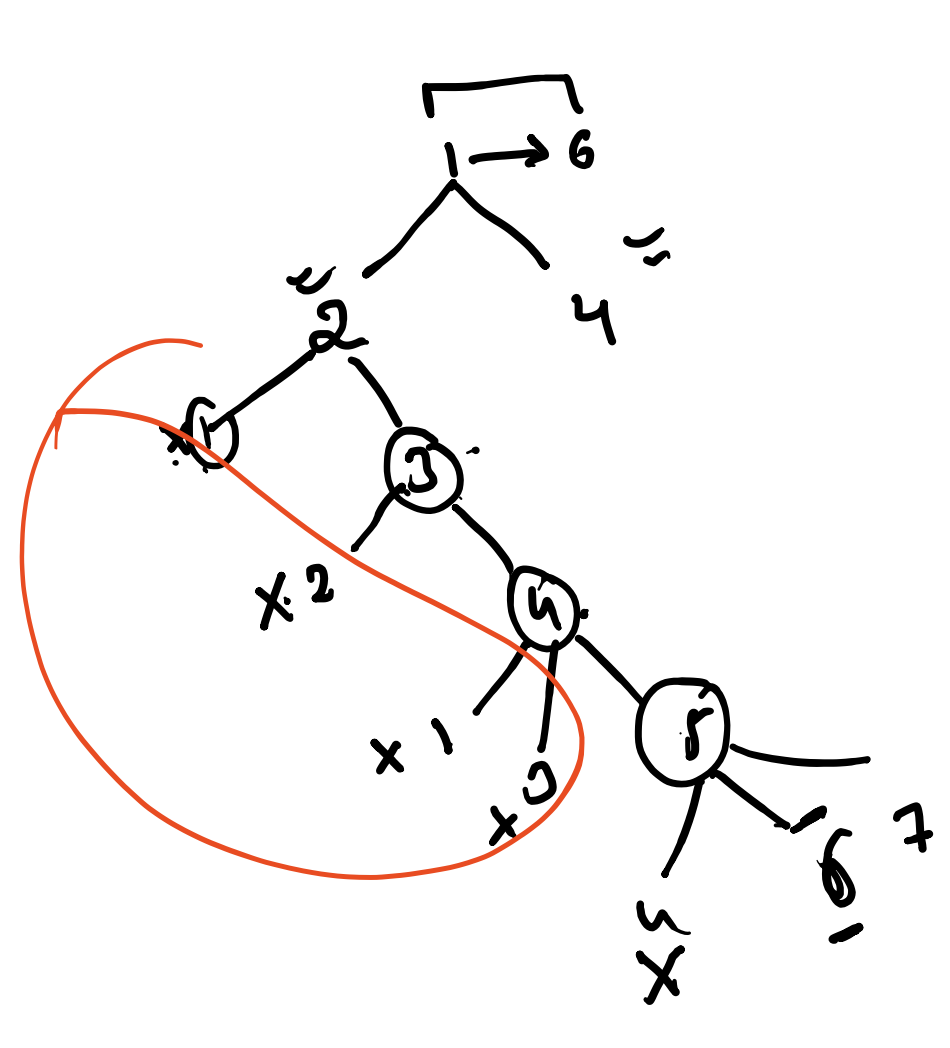
for (int nbrs : map.get(v).keySet()) {

map.get(1).remove(v) 4K (1,3,5)

nbrs

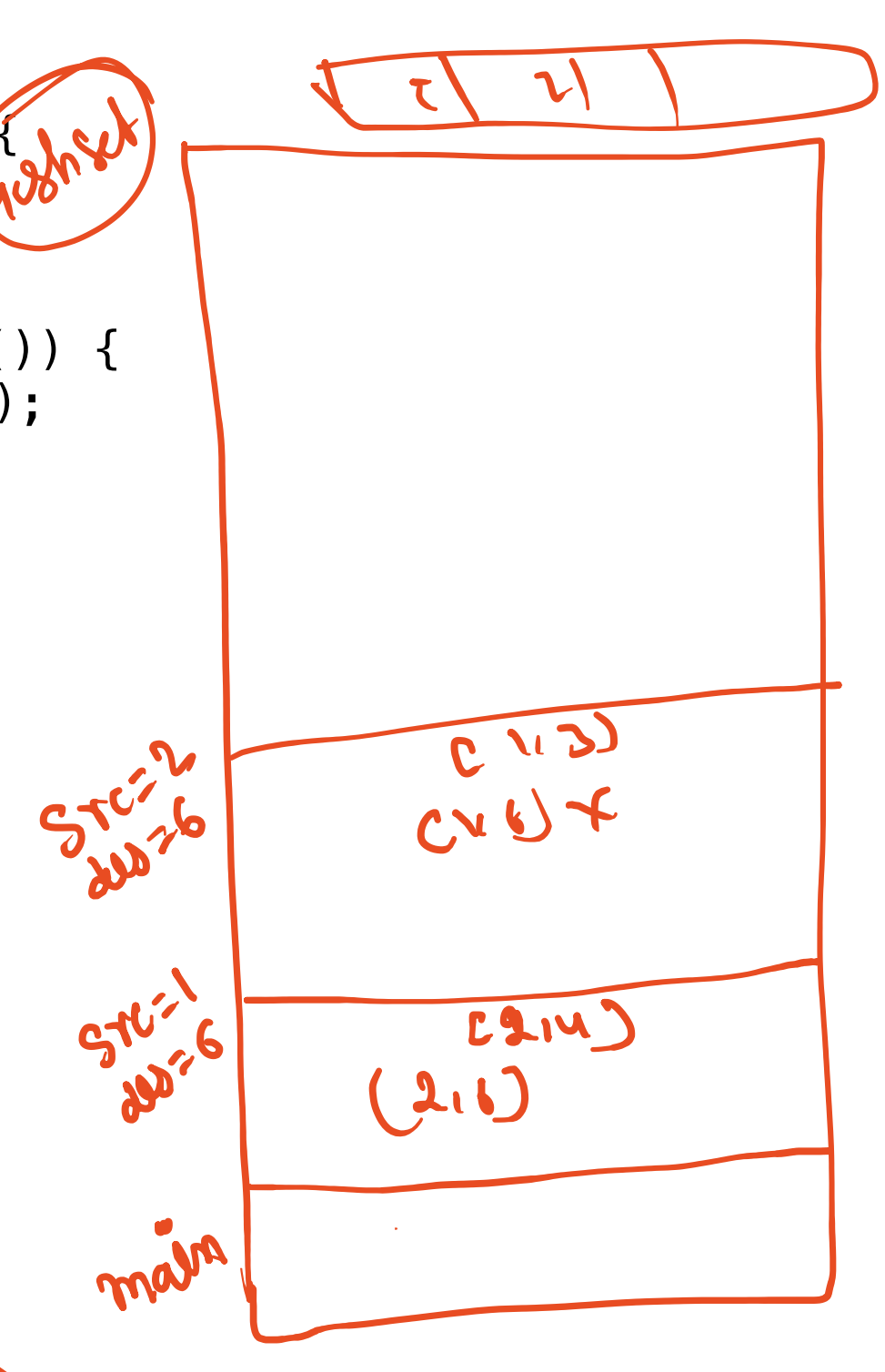

1
2
4
6
5
2
3
5
1
2
3
4
7
6
4

src = 1 des = 6

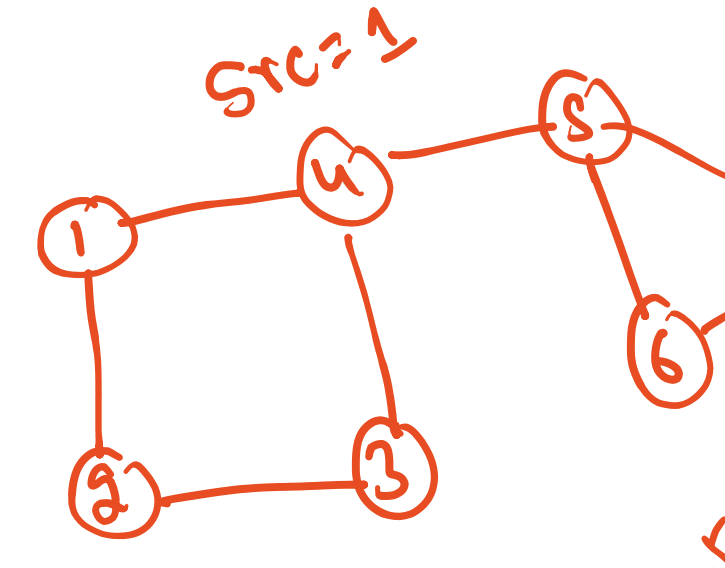

1 → 6
2
4
3
4
5
6
7
x 2
x 1
x 3
x 4

```java
public boolean hashpath(int src, int des) {
        if (src == des) {
            return true;
        }
        for (int nbrs : map.get(src).keySet()) {
            boolean ans = hashpath(nbrs, des);
            if (ans) {
                return true;
            }

        }
        return false;

    }
```

HashSet


1 2
(1,3)
(4,6) x
(2,4)
(2,6)
src=2 des=6
src=1 des=6
main

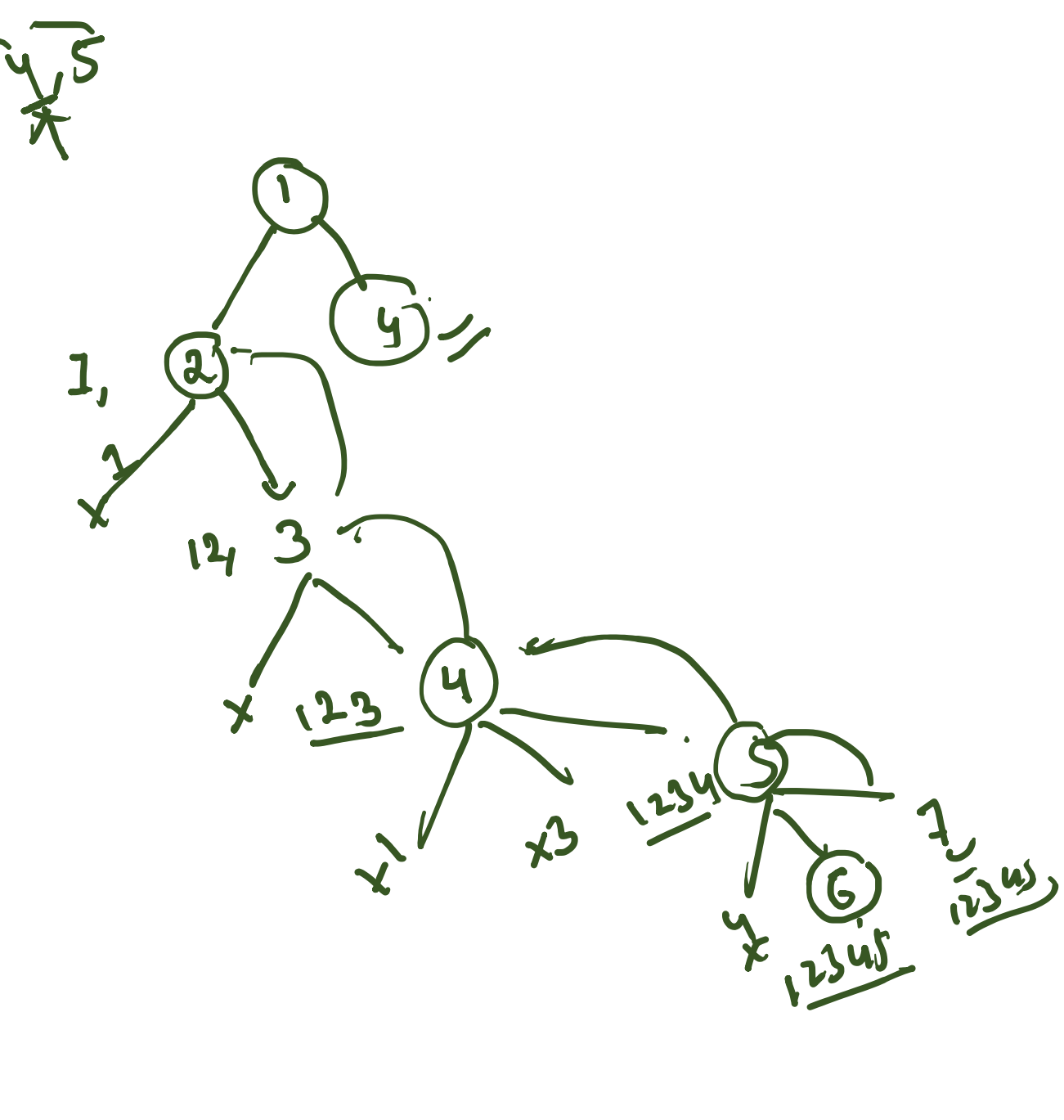
src=1 des=6

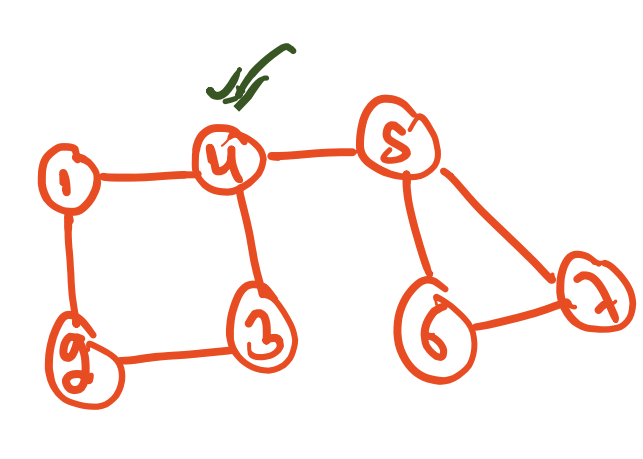


1 2 3 4 5 6
1 2 3 4 5 7 6
1 4 5 6
1 4 5 7 6

```java
public void PrintPath(int src, int des, HashSet<Integer> visited, String ans) {
        if (src == des) {
            System.out.println(ans+des);
            return;
        }
        visited.add(src);
        for (int nbrs : map.get(src).keySet()) {
            if (!visited.contains(nbrs)) {
                PrintPath(nbrs, des, visited, ans + src);
            }

        }
    }
```

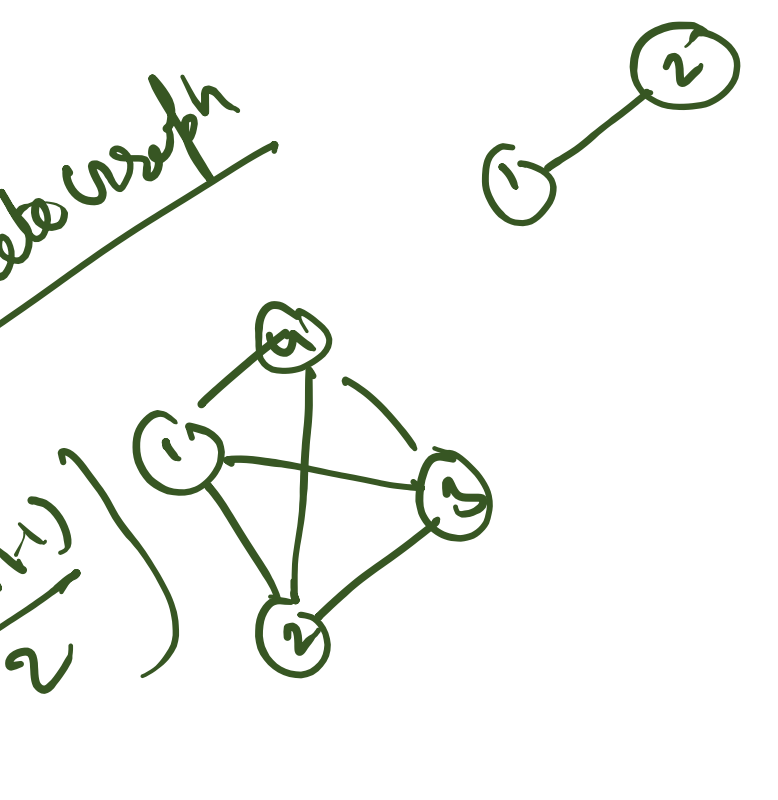
[1, 2, 3, 4, 5]

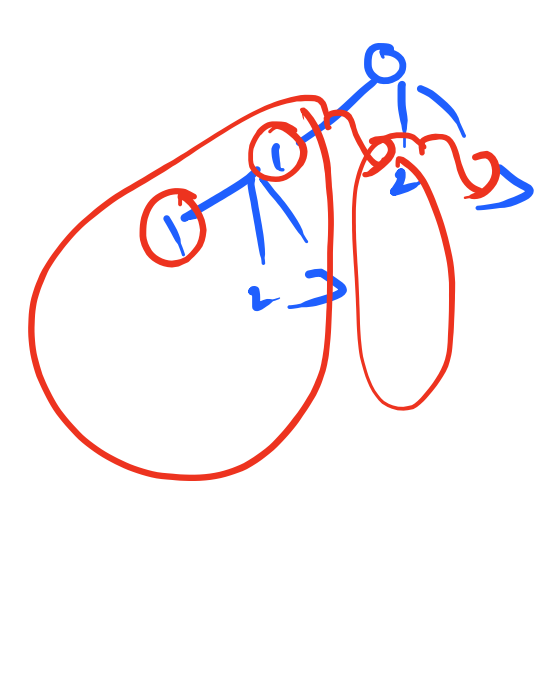

1
4
1,
2
x 1
12, 3
x
123
4
x 1
x3
1234
5
4 x
6
12345
7
1234



Complete Graph

1 → (n-1)
n → n×(n-1)/2

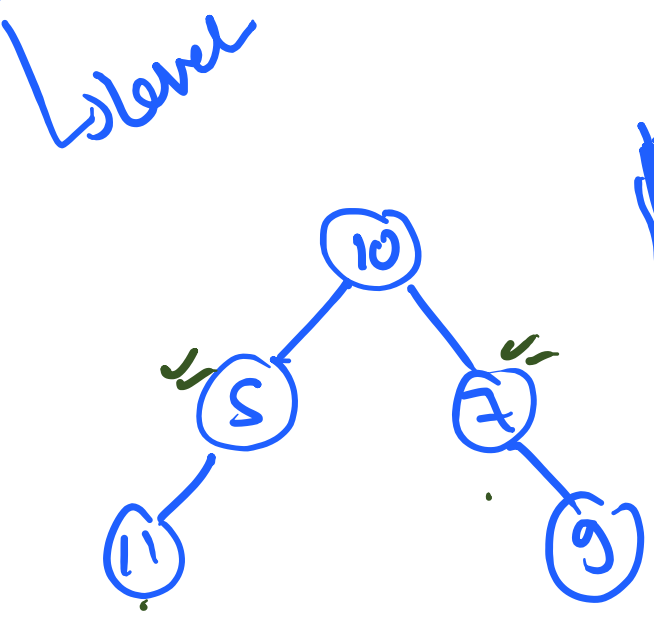




① B.F.S
level

① D.F.S

10 5 7 11 9



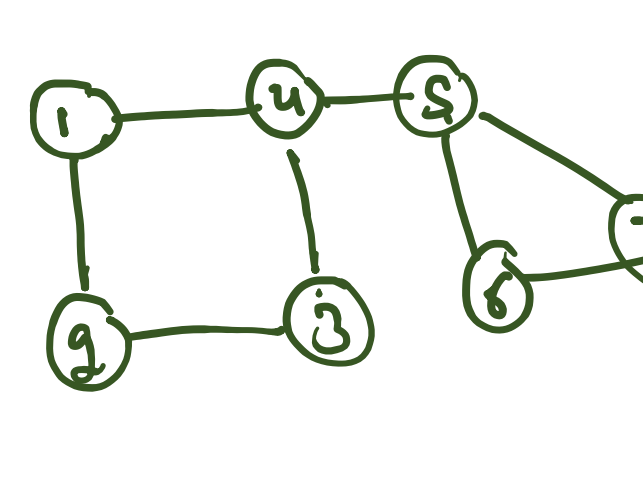
~~10~~|~~5~~|~~7~~|~~11~~|~~9~~

1. remov
2. gg nere
3. marked visited
4. self work
5. Add unvisited nbrs

for(int nbrs: map.get(r).keySet()) (1,3,5)

src=1 des=6

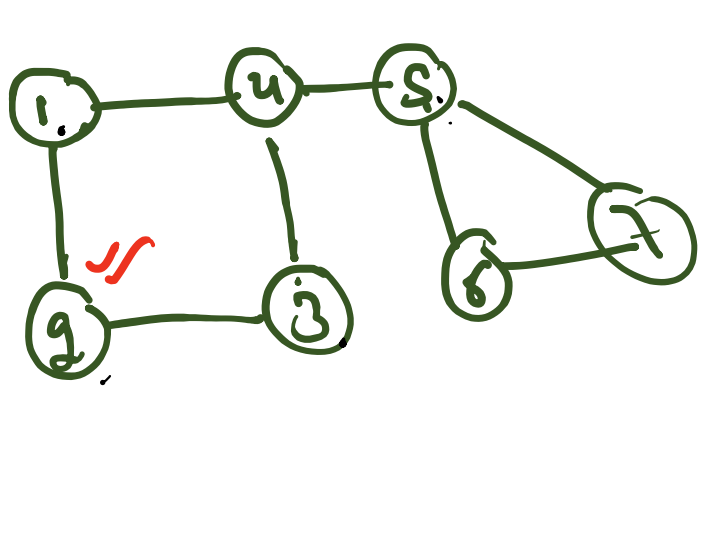
Visited
1 2 4 3
5 6 7



1. remov
2. gg nere →
3. marked visited
4. self work
5. Add unvisited nbrs

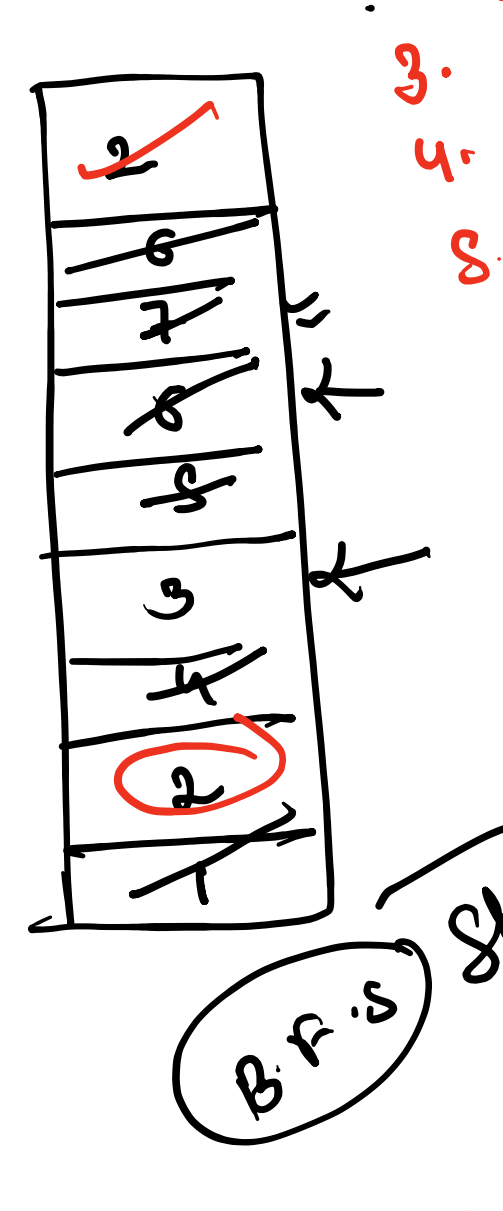
~~1~~|~~2~~|~~4~~|~~3~~|~~3~~|~~5~~|~~6~~|~~7~~|~~7~~

V+E

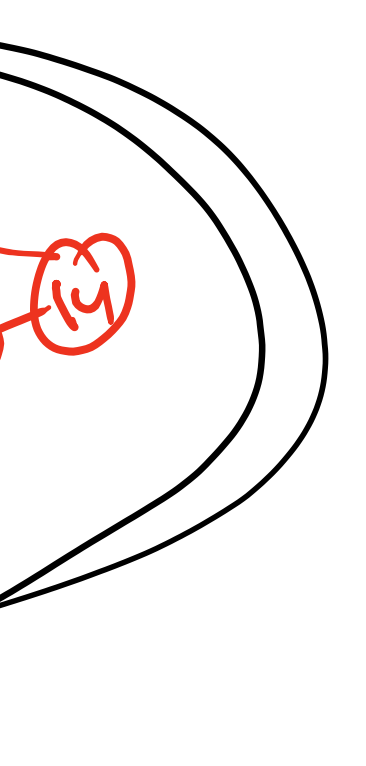


1 4 5 7 6
3 2

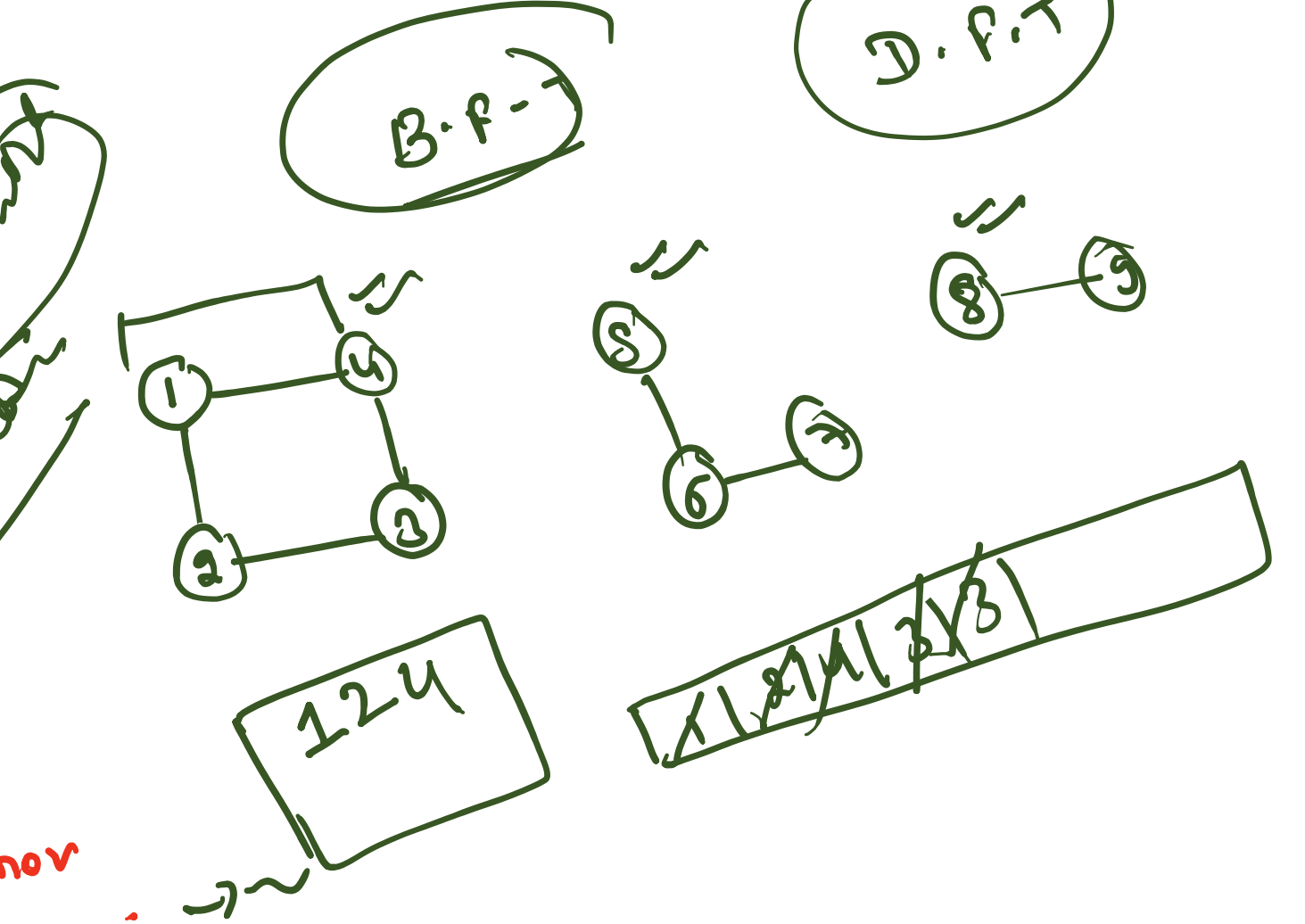
1. remov
2. gg nere → continue
3. marked visited
4. self work
5. Add unvisited nbrs



B.F.S Start



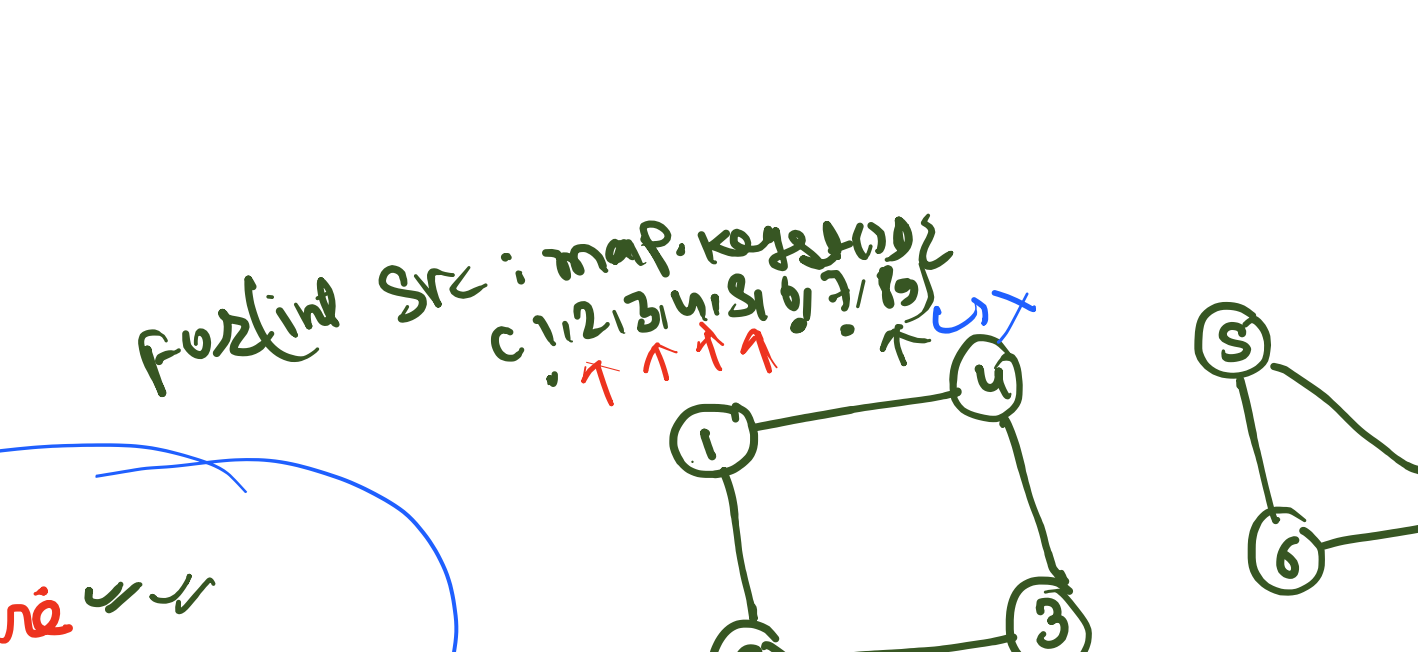
1-3


12
1
1
2
3
4
5
6
7
8
9
10
11
12
13
14

① 3 component disconnected

B.F.T

D.F.T



1 2 4

~~1~~|~~2~~|~~4~~|~~2~~|~~3~~

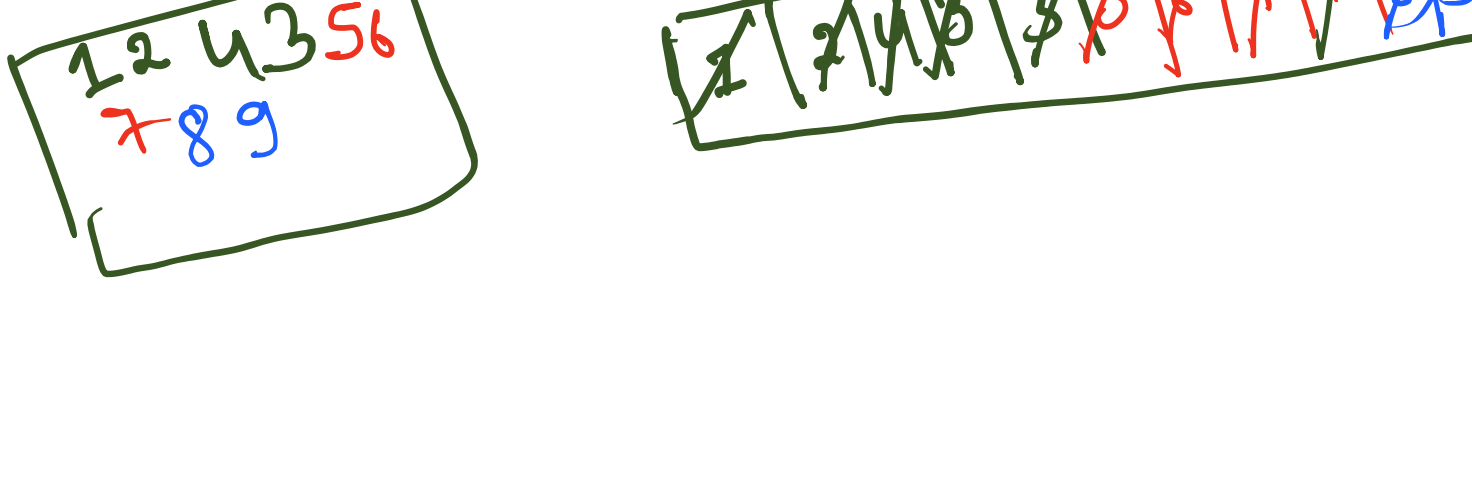
1 2 4 3

1. remov
2. gg nere →
3. marked visited
4. self work → print
5. Add unvisited nbrs

2 3

For(int src : map.keySet()){
[1,2,3,4,5,6,7,8,9]

1. remov
2. gg nere ✓
3. marked visited
4. self work
5. Add unvisited nbrs



1 2 4 3 5 6
7 8 9

1 2 4 3 5 6 7 8 9

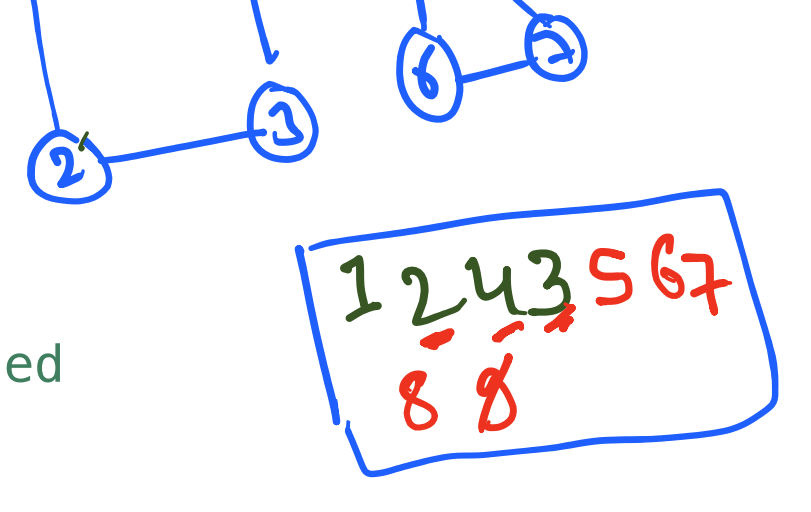
~~1~~|~~2~~|~~4~~|~~2~~|~~3~~|~~3~~|~~5~~|~~6~~|~~7~~|~~7~~|~~8~~|~~9~~

```java
public void BFT() {
        Queue<Integer> q = new LinkedList<>();
        HashSet<Integer> visited = new HashSet<>();
        for (int src : map.keySet()) {
            if (visited.contains(src)) {
                continue;
            }
            // BFS
            q.add(src);
            while (!q.isEmpty()) {
                // 1. remove
                int r = q.poll();
                // 2. Ignore if Already visited
                if (visited.contains(r)) {
                    continue;
                }
                // 3. Marked visted
                visited.add(r);
                // 4. Self work
                System.out.print(r + " ");
                // Add unvisited nbrs
                for (int nbrs : map.get(r).keySet()) {
                    if (!visited.contains(nbrs)) {
                        q.add(nbrs);
                    }
                }
            }
        }
        System.out.println();
    }
```

C=1 2 3

C.H

[1,2,3,4,5,6,7,8,9]



1 2 4 3 5 6 7
8 8

1 2 4 3 5 6 7 8 9

~~1~~|~~2~~|~~4~~|~~3~~|~~3~~|~~5~~|~~6~~|~~6~~|~~7~~|~~7~~|~~8~~|~~9~~